

Модуль поддержки принятия врачебных решений по дозированию варфарина

Распечатка фрагментов исходного кода
(титульный лист оформляется патентоведом)

api/src/Ibs/Context/AICDSS/AiDosage/Service/DosageRecommendationEngine.php
<?php

```
declare(strict_types=1);
```

```
namespace Ibs\Context\AICDSS\AiDosage\Service;
```

```
use Ibs\Context\TreatmentTherapy\Entity\Treatment;  
use Ibs\Context\TreatmentTherapy\Entity\Appointment;  
use Ibs\Context\TreatmentTherapy\Entity\TestHistory;  
use Doctrine\ORM\EntityManagerInterface;
```

```
class DosageRecommendationEngine  
{
```

```
    public function __construct(  
        private EntityManagerInterface $entityManager  
    ) {}
```

```
    public function recommend(int $treatmentId): array  
    {
```

```
        $treatment = $this->entityManager->getRepository(Treatment::class)->find($treatmentId);  
        if (!$treatment) {  
            return ['variants' => [], 'explanation' => 'Лечение не найдено'];  
        }
```

```
        $lastTestHistory = $this->entityManager->getRepository(TestHistory::class)  
            ->findOneBy(['treatment' => $treatment], ['creationDt' => 'DESC']);  
        if (!$lastTestHistory) {  
            return ['variants' => [], 'explanation' => 'Нет данных МНО для расчёта'];  
        }
```

```
        $lastMno = $lastTestHistory->getMno();  
        $mnoFrom = $treatment->getMnoFrom();  
        $mnoTo = $treatment->getMnoTo();
```

```
        // Особые случаи  
        if ($lastMno > 5.0) {  
            return [  
                'variants' => [],  
                'explanation' => 'МНО превышает 5.0. Рекомендуется пропустить приём и связаться с врачом.',  
            ];  
        }  
        if ($lastMno < 1.5) {  
            // Заглушка: проверка механического клапана будет по
```

```

зже
        return [
            'variants' => [],
            'explanation' => 'МНО ниже 1.5. Требуется срочна
я консультация врача.',
        ];
    }

    // Получить текущую дозу (в таблетках)
    $currentDose = $this->getCurrentDose($treatment);
    if ($currentDose === null) {
        return ['variants' => [], 'explanation' => 'Не удало
сь определить текущую дозу'];
    }

    $newDose = $currentDose;
    $explanation = '';

    if ($lastMno < $mnoFrom) {
        $increase = min($currentDose * 0.1, 0.5);
        $newDose = round($currentDose + $increase, 2);
        $explanation = sprintf(
            'МНО = %.2f ниже целевого диапазона (%.2f–%.2f).
Доза увеличена на %.2f таб.',
            $lastMno, $mnoFrom, $mnoTo, $newDose - $currentD
ose
        );
    } elseif ($lastMno > $mnoTo) {
        $decrease = min($currentDose * 0.1, 0.5);
        $newDose = round($currentDose - $decrease, 2);
        $explanation = sprintf(
            'МНО = %.2f выше целевого диапазона (%.2f–%.2f).
Доза уменьшена на %.2f таб.',
            $lastMno, $mnoFrom, $mnoTo, $currentDose - $newD
ose
        );
    } else {
        $newDose = $currentDose;
        $explanation = sprintf(
            'МНО = %.2f в целевом диапазоне (%.2f–%.2f). Доз
а оставлена без изменений.',
            $lastMno, $mnoFrom, $mnoTo
        );
    }

    $newDose = round($newDose * 4) / 4; // округление до 0.2
5

    $variants = [
        ['dose' => max(0, $newDose), 'label' => 'Основной'],

```

```

        ['dose' => max(0, $newDose - 0.25), 'label' => 'Сниж
енный'],
        ['dose' => $newDose + 0.25, 'label' => 'Повышенный']
    ];

    return ['variants' => $variants, 'explanation' => $explanation];
}

private function getCurrentDose(Treatment $treatment): ?float
{
    // Попробуем взять из последнего назначения (appointment)
    $lastAppointment = $this->entityManager->getRepository(A
ppointment::class)
        ->findOneBy(
            ['treatment' => $treatment],
            ['appointmentDt' => 'DESC']
        );

    if ($lastAppointment) {
        return $lastAppointment->getDoze();
    }

    // Если нет назначений, берём из последнего анализа
    $lastTest = $this->entityManager->getRepository(TestHist
ory::class)
        ->findOneBy(
            ['treatment' => $treatment],
            ['creationDt' => 'DESC']
        );

    if ($lastTest) {
        return $lastTest->getDoze();
    }

    return null;
}
}
api/src/Ibs/Context/AICDSS/Pharmacogenetics/Service/PharmacogeneticsServi
<?php

declare(strict_types=1);

namespace Ibs\Context\AICDSS\Pharmacogenetics\Service;

use Ibs\Context\AICDSS\Entity\MarkerDrugRelation;
use Ibs\Context\AICDSS\Entity\PatientGeneticResult;
use Ibs\Context\TreatmentTherapy\Entity\Treatment;

```

```

use Doctrine\ORM\EntityManagerInterface;

class PharmacogeneticsService
{
    public function __construct(
        private EntityManagerInterface $entityManager
    ) {}

    /**
     * @param int $patientId
     * @param int|null $drugId Если передан, фильтрует маркер
     * по препарату через marker_drug_relations.
     * Если null – используется препара
     * т из последнего лечения пациента (старое поведение).
     */
    public function getPatientPharmacogenetics(int $patientId, ?
    int $drugId = null): array
    {
        // Если drugId не передан явно, определяем по последнему
        лечению пациента
        if ($drugId === null) {
            $treatment = $this->getLatestTreatment($patientId);
            if (!$treatment) {
                return [];
            }
            $drug = $treatment->getDrug();
            if (!$drug) {
                return [];
            }
            $drugId = $drug->getId();
        }

        // 2. Получить все связи маркеров с этим препаратом
        $relations = $this->entityManager->getRepository(MarkerD
        rugRelation::class)->findBy([
            'drug' => $drugId,
        ]);

        if (empty($relations)) {
            return [];
        }

        // 3. Собрать маркеры и результаты для пациента
        $resultSet = [];
        foreach ($relations as $relation) {
            $marker = $relation->getMarker();
            if (!$marker) {
                continue;
            }
        }
    }
}

```

```

        // Найти существующий результат для этого пациента и
        маркера
        $existingResult = $this->entityManager->getRepository(
PatientGeneticResult::class)->findOneBy([
            'patient' => $patientId,
            'marker'  => $marker,
        ]);

        $resultSet[] = [
            'markerId'      => $marker->getId(),
            'geneSymbol'    => $marker->getGeneSymbol(),
            'fullName'      => $marker->getFullName(),
            'rsId'          => $marker->getRsId(),
            'possibleValues' => $marker->getPossibleValues()
        ]->map(function ($gmv) {
            return [
                'id'          => $gmv->getId(),
                'value'       => $gmv->getValue(),
                'label'       => $gmv->getLabel(),
                'description' => $gmv->getDescription(),
            ];
        })->toArray(),
        'currentValueId' => $existingResult ? ($existing
Result->getMarkerValue() ? $existingResult->getMarkerValue()
->getId() : null) : null,
        'currentValue'   => $existingResult ? ($existing
Result->getMarkerValue() ? $existingResult->getMarkerValue()
->getValue() : null) : null,
        'resultId'       => $existingResult ? $existingR
esult->getId() : null,
    ];
}

return $resultSet;
}

private function getLatestTreatment(int $patientId): ?Treatm
ent
{
    $repo = $this->entityManager->getRepository(Treatment::c
lass);

    // Сначала ищем активное
    $qb = $repo->createQueryBuilder('t')
        ->where('t.patient = :patientId')
        ->andWhere('t.realEndDt IS NULL')
        ->orderBy('t.begDt', 'DESC')
        ->setMaxResults(1)
        ->setParameter('patientId', $patientId);

```

```

        $treatment = $qb->getQuery()->getOneOrNullResult();

        if ($treatment) {
            return $treatment;
        }

        // Если нет активного – последнее завершённое
        $qb = $repo->createQueryBuilder('t')
            ->where('t.patient = :patientId')
            ->orderBy('t.begDt', 'DESC')
            ->setMaxResults(1)
            ->setParameter('patientId', $patientId);

        return $qb->getQuery()->getOneOrNullResult();
    }
}
api/src/Ibs/Context/LabIoTGateway/Service/PatientVitalsSyncService.php
<?php

declare(strict_types=1);

namespace Ibs\Context\LabIoTGateway\Service;

use Ibs\Context\LabIoTGateway\Entity\PatientVitals;
use Ibs\Context\LabIoTGateway\Entity\PatientVitalsLatest;
use Doctrine\ORM\EntityManagerInterface;

class PatientVitalsSyncService
{
    public function __construct(
        private EntityManagerInterface $entityManager,
    ) {}

    public function syncFromVitals(PatientVitals $vitals): void
    {
        $patient = $vitals->getPatient();
        $latest = $this->entityManager->getRepository(PatientVitalsLatest::class)
            ->findOneBy(['patient' => $patient]);

        if ($latest === null) {
            $latest = new PatientVitalsLatest($patient);
            $this->entityManager->persist($latest);
        }

        if ($vitals->getHb() !== null) {
            $latest->setHb($vitals->getHb());
        }

        if ($vitals->getHeartRate() !== null) {
            $latest->setHeartRate($vitals->getHeartRate());
        }
    }
}

```

```

        }
        if ($vitals->getSystolicPressure() !== null) {
            $latest->setSystolicPressure($vitals->getSystolicPressure());
        }
        if ($vitals->getDiastolicPressure() !== null) {
            $latest->setDiastolicPressure($vitals->getDiastolicPressure());
        }
        if ($vitals->getSaturation() !== null) {
            $latest->setSaturation($vitals->getSaturation());
        }
        if ($vitals->getWeight() !== null) {
            $latest->setWeight($vitals->getWeight());
        }

        $latest->setLastUpdated(new \DateTime());
        $this->entityManager->flush();
    }
}
api/src/Ibs/Context/AICDSS/AiDosage/State/DosageRecommendationProvider.php
<?php

declare(strict_types=1);

namespace Ibs\Context\AICDSS\AiDosage\State;

use ApiPlatform\Metadata\Operation;
use ApiPlatform\State\ProviderInterface;
use Ibs\Context\AICDSS\AiDosage\Dto\DosageRecommendation;
use Ibs\Context\AICDSS\AiDosage\Service\DosageRecommendationEngine;
use Symfony\Component\HttpFoundation\RequestStack;

class DosageRecommendationProvider implements ProviderInterface
{
    public function __construct(
        private DosageRecommendationEngine $engine,
        private RequestStack $requestStack
    ) {}

    public function provide(Operation $operation, array $uriVariables = [], array $context = []): DosageRecommendation
    {
        $request = $this->requestStack->getCurrentRequest();
        $treatmentId = (int) $request->query->get('treatment_id');

        $result = $this->engine->recommend($treatmentId);
    }
}

```

```

        $dto = new DosageRecommendation();
        $dto->variants = $result['variants'];
        $dto->explanation = $result['explanation'];

        return $dto;
    }
}
api/src/Ibs/Context/AICDSS/Pharmacogenetics/State/PatientGeneticResultProcessor.php
<?php

declare(strict_types=1);

namespace Ibs\Context\AICDSS\Pharmacogenetics\State;

use ApiPlatform\Metadata\Operation;
use ApiPlatform\Metadata\Post as PostOperation;
use ApiPlatform\Metadata\Put as PutOperation;
use ApiPlatform\State\ProcessorInterface;
use Ibs\Context\AICDSS\Entity\GeneticMarker;
use Ibs\Context\AICDSS\Entity\GeneticMarkerValue;
use Ibs\Context\AICDSS\Entity\PatientGeneticResult;
use Doctrine\ORM\EntityManagerInterface;
use Symfony\Component\DependencyInjection\Attribute\Autowire;
use Symfony\Component\HttpKernel\Exception\BadRequestHttpException;
use Symfony\Component\HttpKernel\Exception\ConflictHttpException;
use Symfony\Component\HttpKernel\Exception\NotFoundHttpException;
use Symfony\Component\Security\Core\Authentication\Token\Storage\TokenStorageInterface;
use Symfony\Component\Security\Core\User\UserInterface;
use Symfony\Contracts\Translation\TranslatorInterface;

class PatientGeneticResultProcessor implements ProcessorInterface
{
    public function __construct(
        #[Autowire('@api_platform.doctrine.orm.state.persist_processor')]
        private ProcessorInterface $persistProcessor,
        private EntityManagerInterface $entityManager,
        private TranslatorInterface $translator,
        private TokenStorageInterface $tokenStorage
    ) {}

    public function process(mixed $data, Operation $operation, array $uriVariables = [], array $context = []): PatientGeneticResult
    {

```

```

/** @var PatientGeneticResult $data */

// ----- 1. Marker -----
$markerInput = $data->getMarker();
if ($markerInput === null) {
    throw new NotFoundException(
        $this->translator->trans('genetic_result.marker_
required', [], 'validators')
    );
}
$markerId = $markerInput->getId();
if ($markerId === null) {
    throw new NotFoundException(
        $this->translator->trans('genetic_result.marker_
id_required', [], 'validators')
    );
}
$marker = $this->entityManager->find(GeneticMarker::clas
s, $markerId);
if ($marker === null) {
    throw new NotFoundException(
        $this->translator->trans('genetic_result.marker_
not_found', ['%markerId%' => $markerId], 'validators')
    );
}
$data->setMarker($marker);

// ----- 2. MarkerValue -----
$valueInput = $data->getMarkerValue();
if ($valueInput === null) {
    throw new NotFoundException(
        $this->translator->trans('genetic_result.marker_
value_required', [], 'validators')
    );
}
$valueId = $valueInput->getId();
if ($valueId === null) {
    throw new NotFoundException(
        $this->translator->trans('genetic_result.marker_
value_id_required', [], 'validators')
    );
}
$value = $this->entityManager->find(GeneticMarkerValue::
class, $valueId);
if ($value === null) {
    throw new NotFoundException(
        $this->translator->trans('genetic_result.marker_
value_not_found', ['%valueId%' => $valueId], 'validators')
    );
}

```

```

        // Принадлежность значения маркеру
        if ($value->getMarker() === null || $value->getMarker()-
>getId() !== $marker->getId()) {
            throw new BadRequestHttpException(
                $this->translator->trans(
                    'genetic_result.marker_value_wrong_marker',
                    ['%valueId%' => $valueId, '%markerId%' => $m
arker->getId()],
                    'validators'
                )
            );
        }
        $data->setMarkerValue($value);

        // ----- 3. Проверка дубликата (только для POST) --
        -----
        if ($operation instanceof PostOperation) {
            $patient = $data->getPatient();
            if ($patient === null) {
                throw new BadRequestHttpException(
                    $this->translator->trans('genetic_result.pat
ient_required', [], 'validators')
                );
            }

            $existing = $this->entityManager->getRepository(Pati
entGeneticResult::class)->findOneBy([
                'patient' => $patient,
                'marker' => $marker,
            ]);

            if ($existing !== null) {
                throw new ConflictHttpException(
                    $this->translator->trans('genetic_result.dup
licate_result', [], 'validators')
                );
            }
        }

        // ----- 4. Аудит: заполнение created_by / updated_
by -----
        $username = $this->getCurrentUsername();

        if ($operation instanceof PostOperation) {
            $data->setCreatedBy($username);
            $data->setUpdatedBy($username);
        } elseif ($operation instanceof PutOperation) {
            // createdBy не трогаем – он уже сохранён при создан
ии и защищён #[Ignore]

```

```

        $data->setUpdatedBy($username);
        $data->setUpdatedAt(new \DateTime());
    }

    // ----- 5. Передача стандартному процессору -----
    -----
    return $this->persistProcessor->process($data, $operation, $uriVariables, $context);
}

private function getCurrentUsername(): ?string
{
    $token = $this->tokenStorage->getToken();
    if (null === $token) {
        return null;
    }
    $user = $token->getUser();
    return ($user instanceof UserInterface) ? $user->getUserIdentifier() : null;
}
}
api/src/Ibs/Context/AICDSS/Pharmacogenetics/State/PharmacogeneticsProvider.php
<?php

declare(strict_types=1);

namespace Ibs\Context\AICDSS\Pharmacogenetics\State;

use ApiPlatform\Metadata\Operation;
use ApiPlatform\State\ProviderInterface;
use Ibs\Context\AICDSS\Pharmacogenetics\Dto\PharmacogeneticsResponse;
use Ibs\Context\AICDSS\Pharmacogenetics\Service\PharmacogeneticsService;
use Symfony\Component\HttpFoundation\Request;
use Symfony\Component\HttpFoundation\Exception\BadRequestHttpException;

class PharmacogeneticsProvider implements ProviderInterface
{
    public function __construct(
        private PharmacogeneticsService $service
    ) {}

    public function provide(Operation $operation, array $uriVariables = [], array $context = []): PharmacogeneticsResponse
    {
        $patientId = (int) $uriVariables['patientId'];

        // Извлекаем drug IRI из query-параметров

```

```

        $request = $context['request'] ?? null;
        $drugId = null;

        if ($request instanceof Request) {
            $drugIri = $request->query->get('drug');
            if ($drugIri !== null) {
                $drugId = $this->extractIdFromIri($drugIri);
                if ($drugId === null) {
                    throw new BadRequestHttpException('Неверный
формат IRI для параметра drug.');
                }
            }
        }

        $response = new PharmacogeneticsResponse();
        $response->markers = $this->service->getPatientPharmacog
enetics($patientId, $drugId);

        return $response;
    }

    /**
     * Извлекает числовой ID из IRI вида /api/drugs/123.
     *
     * @param string $iri
     * @return int|null
     */
    private function extractIdFromIri(string $iri): ?int
    {
        if (preg_match('~/(\\d+)$~', $iri, $matches)) {
            return (int) $matches[1];
        }
        return null;
    }
}
api/src/Ibs/Context/LabIoTGateway/State/PatientVitalsLatestBatchProvider.
<?php
declare(strict_types=1);

namespace Ibs\\Context\\LabIoTGateway\\State;

use ApiPlatform\\Metadata\\Operation;
use ApiPlatform\\State\\ProviderInterface;
use Ibs\\Context\\LabIoTGateway\\Entity\\PatientVitalsLatest;
use Doctrine\\ORM\\EntityManagerInterface;
use Symfony\\Component\\HttpFoundation\\Request;

class PatientVitalsLatestBatchProvider implements ProviderInterf
ace
{

```

```

public function __construct(private EntityManagerInterface $
em) {}

public function provide(Operation $operation, array $uriVari
ables = [], array $context = []): iterable
{
    $request = $context['request'] ?? null;
    $qb = $this->em->getRepository(PatientVitalsLatest::clas
s)->createQueryBuilder('v');

    if ($request instanceof Request) {
        $patientIds = $request->query->all('patient_id');
        if (!empty($patientIds)) {
            // Приводим значения к целым числам
            $patientIds = array_map('intval', $patientIds);
            $qb->where($qb->expr()->in('v.patient', ':ids'))
                ->setParameter('ids', $patientIds);
        }
    }

    return $qb->getQuery()->getResult();
}
}

```

api/src/Ibs/Context/AICDSS/Entity/GeneticMarker.php
<?php

```

declare(strict_types=1);

namespace Ibs\Context\AICDSS\Entity;

use Doctrine\ORM\Mapping as ORM;
use Doctrine\Common\Collections\ArrayCollection;
use Doctrine\Common\Collections\Collection;
use ApiPlatform\Metadata\ApiResource;
use ApiPlatform\Metadata\Get;

#[ApiResource(
    operations: [
        new Get()
    ]
)]
#[ORM\Entity]
#[ORM\Table(name: 'genetic_markers')]
class GeneticMarker
{
    #[ORM\Id]
    #[ORM\GeneratedValue(strategy: 'IDENTITY')]
    #[ORM\Column(type: 'integer')]
    private ?int $id = null;
}

```

```

#[ORM\Column(type: 'string', length: 30)]
private string $geneSymbol;

#[ORM\Column(type: 'string', length: 150)]
private string $fullName;

#[ORM\Column(type: 'string', length: 50, nullable: true)]
private ?string $rsId = null;

#[ORM\OneToMany(targetEntity: GeneticMarkerValue::class, mappedBy: 'marker', cascade: ['persist'])]
private Collection $possibleValues;

#[ORM\Column(type: 'text', nullable: true)]
private ?string $description = null;

public function __construct()
{
    $this->possibleValues = new ArrayCollection();
}

public function getId(): ?int
{
    return $this->id;
}

public function getGeneSymbol(): string
{
    return $this->geneSymbol;
}

public function setGeneSymbol(string $geneSymbol): self
{
    $this->geneSymbol = $geneSymbol;
    return $this;
}

public function getFullName(): string
{
    return $this->fullName;
}

public function setFullName(string $fullName): self
{
    $this->fullName = $fullName;
    return $this;
}

public function getRsId(): ?string
{

```

```

        return $this->rsId;
    }

    public function setRsId(?string $rsId): self
    {
        $this->rsId = $rsId;
        return $this;
    }

    public function getPossibleValues(): Collection
    {
        return $this->possibleValues;
    }

    public function addPossibleValue(GeneticMarkerValue $value):
    self
    {
        if (!$this->possibleValues->contains($value)) {
            $this->possibleValues->add($value);
            $value->setMarker($this);
        }
        return $this;
    }

    public function removePossibleValue(GeneticMarkerValue $valu
e): self
    {
        if ($this->possibleValues->contains($value)) {
            $this->possibleValues->removeElement($value);
            if ($value->getMarker() === $this) {
                $value->setMarker(null);
            }
        }
        return $this;
    }

    public function getDescription(): ?string
    {
        return $this->description;
    }

    public function setDescription(?string $description): self
    {
        $this->description = $description;
        return $this;
    }
}
api/src/Ibs/Context/AICDSS/Entity/GeneticMarkerValue.php
<?php

```

```

declare(strict_types=1);

namespace Ibs\Context\AICDSS\Entity;

use Doctrine\ORM\Mapping as ORM;
use ApiPlatform\Metadata\ApiResource;
use ApiPlatform\Metadata\Get;

#[ApiResource(
    operations: [
        new Get()
    ]
)]
#[ORM\Entity]
#[ORM\Table(name: 'genetic_marker_values')]
class GeneticMarkerValue
{
    #[ORM\Id]
    #[ORM\GeneratedValue(strategy: 'IDENTITY')]
    #[ORM\Column(type: 'integer')]
    private ?int $id = null;

    #[ORM\ManyToOne(targetEntity: GeneticMarker::class, inversed
    By: 'possibleValues')]
    #[ORM\JoinColumn(name: 'marker_id', referencedColumnName: 'i
    d', nullable: false)]
    private ?GeneticMarker $marker = null;

    #[ORM\Column(type: 'string', length: 50)]
    private string $value;

    #[ORM\Column(type: 'string', length: 255)]
    private string $label;

    #[ORM\Column(type: 'text', nullable: true)]
    private ?string $description = null;

    public function getId(): ?int
    {
        return $this->id;
    }

    public function getMarker(): ?GeneticMarker
    {
        return $this->marker;
    }

    public function setMarker(?GeneticMarker $marker): self
    {
        $this->marker = $marker;
    }
}

```

```

        return $this;
    }

    public function getValue(): string
    {
        return $this->value;
    }

    public function setValue(string $value): self
    {
        $this->value = $value;
        return $this;
    }

    public function getLabel(): string
    {
        return $this->label;
    }

    public function setLabel(string $label): self
    {
        $this->label = $label;
        return $this;
    }

    public function getDescription(): ?string
    {
        return $this->description;
    }

    public function setDescription(?string $description): self
    {
        $this->description = $description;
        return $this;
    }
}

```

api/src/Ibs/Context/AICDSS/Entity/MarkerDrugRelation.php

```
<?php
```

```
declare(strict_types=1);
```

```
namespace Ibs\Context\AICDSS\Entity;
```

```
use Ibs\Context\TreatmentTherapy\Entity\Drug;
```

```
use Doctrine\ORM\Mapping as ORM;
```

```
#[ORM\Entity]
```

```
#[ORM\Table(name: 'marker_drug_relations')]
```

```
#[ORM\UniqueConstraint(name: 'uniq_marker_drug', columns: ['marker_id', 'drug_id'])]
```

```

class MarkerDrugRelation
{
    #[ORM\Id]
    #[ORM\GeneratedValue(strategy: 'IDENTITY')]
    #[ORM\Column(type: 'integer')]
    private ?int $id = null;

    #[ORM\ManyToOne(targetEntity: GeneticMarker::class)]
    #[ORM\JoinColumn(name: 'marker_id', referencedColumnName: 'id', nullable: false)]
    private ?GeneticMarker $marker = null;

    #[ORM\ManyToOne(targetEntity: Drug::class)]
    #[ORM\JoinColumn(name: 'drug_id', referencedColumnName: 'id', nullable: false)]
    private ?Drug $drug = null;

    public function getId(): ?int
    {
        return $this->id;
    }

    public function getMarker(): ?GeneticMarker
    {
        return $this->marker;
    }

    public function setMarker(?GeneticMarker $marker): self
    {
        $this->marker = $marker;
        return $this;
    }

    public function getDrug(): ?Drug
    {
        return $this->drug;
    }

    public function setDrug(?Drug $drug): self
    {
        $this->drug = $drug;
        return $this;
    }
}

```

api/src/Ibs/Context/AICDSS/Entity/PatientGeneticResult.php
 <?php

declare(strict_types=1);

namespace Ibs\Context\AICDSS\Entity;

```

use ApiPlatform\Doctrine\Orm\Filter\SearchFilter;
use ApiPlatform\Metadata\ApiFilter;
use ApiPlatform\Metadata\ApiResource;
use ApiPlatform\Metadata\Delete;
use ApiPlatform\Metadata\Get;
use ApiPlatform\Metadata\GetCollection;
use ApiPlatform\Metadata\Post;
use ApiPlatform\Metadata\Put;
use Ibs\Context\AICDSS\Pharmacogenetics\State\PatientGeneticResultProcessor;
use Ibs\Context\PatientManagement\Entity\Patient;
use Doctrine\ORM\Mapping as ORM;
use Symfony\Component\Serializer\Annotation\Ignore;

#[ApiResource(
    operations: [
        new GetCollection(),
        new Post(processor: PatientGeneticResultProcessor::class),
    ],
    new Get(),
    new Put(processor: PatientGeneticResultProcessor::class),
    new Delete(),
)]
#[ORM\Entity]
#[ORM\Table(name: 'patient_genetic_results')]
#[ORM\UniqueConstraint(name: 'uniq_patient_marker', columns: ['patient_id', 'marker_id'])]
#[ApiFilter(SearchFilter::class, properties: ['patient' => 'exact'])]
class PatientGeneticResult
{
    #[ORM\Id]
    #[ORM\GeneratedValue(strategy: 'IDENTITY')]
    #[ORM\Column(type: 'integer')]
    private ?int $id = null;

    #[ORM\ManyToOne(targetEntity: Patient::class)]
    #[ORM\JoinColumn(name: 'patient_id', referencedColumnName: 'id', nullable: false, onDelete: 'CASCADE')]
    private ?Patient $patient = null;

    #[ORM\ManyToOne(targetEntity: GeneticMarker::class)]
    #[ORM\JoinColumn(name: 'marker_id', referencedColumnName: 'id', nullable: false)]
    private ?GeneticMarker $marker = null;

    #[ORM\ManyToOne(targetEntity: GeneticMarkerValue::class)]

```

```

#[ORM\JoinColumn(name: 'marker_value_id', referencedColumnName: 'id', nullable: false)]
private ?GeneticMarkerValue $markerValue = null;

#[ORM\Column(type: 'datetime')]
private \DateTimeInterface $createdAt;

#[ORM\Column(type: 'datetime')]
private \DateTimeInterface $updatedAt;

#[ORM\Column(type: 'string', length: 255, nullable: true)]
#[Ignore] // не принимать от клиента
private ?string $createdBy = null;

#[ORM\Column(type: 'string', length: 255, nullable: true)]
#[Ignore] // не принимать от клиента
private ?string $updatedBy = null;

public function __construct()
{
    $this->createdAt = new \DateTime();
    $this->updatedAt = new \DateTime();
}

public function getId(): ?int
{
    return $this->id;
}

public function getPatient(): ?Patient
{
    return $this->patient;
}

public function setPatient(?Patient $patient): self
{
    $this->patient = $patient;

    return $this;
}

public function getMarker(): ?GeneticMarker
{
    return $this->marker;
}

public function setMarker(?GeneticMarker $marker): self
{
    $this->marker = $marker;
}

```

```

        return $this;
    }

    public function getCreatedAt(): \DateTimeInterface
    {
        return $this->createdAt;
    }

    public function setCreatedAt(\DateTimeInterface $createdAt):
        self
    {
        $this->createdAt = $createdAt;

        return $this;
    }

    public function getUpdatedAt(): \DateTimeInterface
    {
        return $this->updatedAt;
    }

    public function setUpdatedAt(\DateTimeInterface $updatedAt):
        self
    {
        $this->updatedAt = $updatedAt;

        return $this;
    }

    public function getMarkerValue(): ?GeneticMarkerValue
    {
        return $this->markerValue;
    }

    public function setMarkerValue(?GeneticMarkerValue $markerVa
        lue): self
    {
        $this->markerValue = $markerValue;
        return $this;
    }

    public function getCreatedBy(): ?string
    {
        return $this->createdBy;
    }

    public function setCreatedBy(?string $createdBy): self
    {
        $this->createdBy = $createdBy;
        return $this;
    }

```

```

    }

    public function getUpdatedBy(): ?string
    {
        return $this->updatedBy;
    }

    public function setUpdatedBy(?string $updatedBy): self
    {
        $this->updatedBy = $updatedBy;
        return $this;
    }
}

```

api/src/Ibs/Context/LabIoTGateway/Entity/PatientVitals.php

```

<?php

declare(strict_types=1);

namespace Ibs\Context\LabIoTGateway\Entity;

use ApiPlatform\Metadata\ApiResource;
use ApiPlatform\Metadata\Get;
use ApiPlatform\Metadata\GetCollection;
use ApiPlatform\Metadata\Post;
use ApiPlatform\Metadata\Put;
use Ibs\Context\PatientManagement\Entity\Patient;
use Ibs\Context\TreatmentTherapy\Entity\Treatment;
use Doctrine\ORM\Mapping as ORM;
use Symfony\Component\Serializer\Annotation\Groups;
use Symfony\Component\Validator\Constraints as Assert;
use Ibs\Context\LabIoTGateway\Validator\Constraints\AtLeastOneVitalSign;
use Symfony\Component\Serializer\Annotation\Ignore;
use ApiPlatform\Doctrine\Orm\FILTER\SearchFilter;
use ApiPlatform\Metadata\ApiFilter;

#[ApiResource(
    operations: [
        new GetCollection(),
        new Post(),
        new Get(),
        new Put(),
    ],
    normalizationContext: ['groups' => ['vitals:read']],
    denormalizationContext: ['groups' => ['vitals:write']],
)]
#[ApiFilter(SearchFilter::class, properties: ['treatment' => 'exact'])]
#[ORM\Entity]
#[ORM\Table(name: 'patient_vitals')]

```

```

#[ORM\Index(
    name: 'idx_vitals_patient_record',
    columns: ['patient_id', 'record_dt'],
    options: ['order' => ['record_dt' => 'DESC']]
)]
#[ORM\HasLifecycleCallbacks]
#[AtLeastOneVitalSign]
class PatientVitals
{
    #[ORM\Id]
    #[ORM\GeneratedValue(strategy: 'IDENTITY')]
    #[ORM\Column(type: 'integer')]
    #[Groups(['vitals:read'])]
    private ?int $id = null;

    #[ORM\ManyToOne(targetEntity: Patient::class)]
    #[ORM\JoinColumn(name: 'patient_id', referencedColumnName: 'id', nullable: false, onDelete: 'CASCADE')]
    #[Assert\NotNull]
    #[Groups(['vitals:read', 'vitals:write'])]
    private ?Patient $patient = null;

    #[ORM\ManyToOne(targetEntity: Treatment::class)]
    #[ORM\JoinColumn(name: 'treatment_id', referencedColumnName: 'id', nullable: true, onDelete: 'SET NULL')]
    #[Groups(['vitals:read', 'vitals:write'])]
    private ?Treatment $treatment = null;

    #[ORM\Column(type: 'datetime')]
    #[Assert\NotBlank]
    #[Assert\Type(\DateTimeInterface::class)]
    #[Groups(['vitals:read', 'vitals:write'])]
    private \DateTimeInterface $recordDt;

    #[ORM\Column(type: 'float', nullable: true)]
    #[Assert\Positive]
    #[Assert\Range(min: 50, max: 250)]
    #[Groups(['vitals:read', 'vitals:write'])]
    private ?float $hb = null;

    #[ORM\Column(type: 'integer', nullable: true)]
    #[Assert\Positive]
    #[Assert\Range(min: 30, max: 250)]
    #[Groups(['vitals:read', 'vitals:write'])]
    private ?int $heartRate = null;

    #[ORM\Column(type: 'integer', nullable: true)]
    #[Assert\Positive]
    #[Assert\Range(min: 70, max: 250)]
    #[Groups(['vitals:read', 'vitals:write'])]

```

```

private ?int $systolicPressure = null;

#[ORM\Column(type: 'integer', nullable: true)]
#[Assert\Positive]
#[Assert\Range(min: 70, max: 250)]
#[Groups(['vitals:read', 'vitals:write'])]
private ?int $diastolicPressure = null;

#[ORM\Column(type: 'integer', nullable: true)]
#[Assert\Positive]
#[Assert\Range(min: 50, max: 100)]
#[Groups(['vitals:read', 'vitals:write'])]
private ?int $saturation = null;

#[ORM\Column(type: 'text', nullable: true)]
#[Groups(['vitals:read', 'vitals:write'])]
private ?string $comment = null;

#[ORM\Column(type: 'datetime')]
#[Groups(['vitals:read'])]
private \DateTimeInterface $createdAt;

#[ORM\Column(type: 'datetime')]
#[Groups(['vitals:read'])]
private \DateTimeInterface $updatedAt;

#[ORM\Column(type: 'string', length: 255, nullable: true)]
#[Ignore]
private ?string $createdBy = null;

#[ORM\Column(type: 'string', length: 255, nullable: true)]
#[Ignore]
private ?string $updatedBy = null;

#[ORM\Column(type: 'float', nullable: true)]
#[Assert\Positive]
#[Assert\Range(min: 20, max: 300)]
#[Groups(['vitals:read', 'vitals:write'])]
private ?float $weight = null;

public function __construct()
{
    $this->recordDt = new \DateTime();
    $this->createdAt = new \DateTime();
    $this->updatedAt = new \DateTime();
}

#[ORM\PreUpdate]
public function onPreUpdate(): void
{

```

```

        $this->updatedAt = new \DateTime();
    }

    public function getId(): ?int
    {
        return $this->id;
    }

    public function getPatient(): ?Patient
    {
        return $this->patient;
    }

    public function setPatient(?Patient $patient): self
    {
        $this->patient = $patient;
        return $this;
    }

    public function getTreatment(): ?Treatment
    {
        return $this->treatment;
    }

    public function setTreatment(?Treatment $treatment): self
    {
        $this->treatment = $treatment;
        return $this;
    }

    public function getRecordDt(): \DateTimeInterface
    {
        return $this->recordDt;
    }

    public function setRecordDt(\DateTimeInterface $recordDt): self
    {
        $this->recordDt = $recordDt;
        return $this;
    }

    public function getHb(): ?float
    {
        return $this->hb;
    }

    public function setHb(?float $hb): self
    {
        $this->hb = $hb;
    }

```

```

        return $this;
    }

    public function getHeartRate(): ?int
    {
        return $this->heartRate;
    }

    public function setHeartRate(?int $heartRate): self
    {
        $this->heartRate = $heartRate;
        return $this;
    }

    public function getSystolicPressure(): ?int
    {
        return $this->systolicPressure;
    }

    public function setSystolicPressure(?int $systolicPressure):
    self
    {
        $this->systolicPressure = $systolicPressure;
        return $this;
    }

    public function getDiastolicPressure(): ?int
    {
        return $this->diastolicPressure;
    }

    public function setDiastolicPressure(?int $diastolicPressure
    ): self
    {
        $this->diastolicPressure = $diastolicPressure;
        return $this;
    }

    public function getSaturation(): ?int
    {
        return $this->saturation;
    }

    public function setSaturation(?int $saturation): self
    {
        $this->saturation = $saturation;
        return $this;
    }

    public function getComment(): ?string

```

```

{
    return $this->comment;
}

public function setComment(?string $comment): self
{
    $this->comment = $comment;
    return $this;
}

public function getCreatedAt(): \DateTimeInterface
{
    return $this->createdAt;
}

public function getUpdatedAt(): \DateTimeInterface
{
    return $this->updatedAt;
}

public function getCreatedBy(): ?string
{
    return $this->createdBy;
}

public function setCreatedBy(?string $createdBy): self
{
    $this->createdBy = $createdBy;
    return $this;
}

public function getUpdatedBy(): ?string
{
    return $this->updatedBy;
}

public function setUpdatedBy(?string $updatedBy): self
{
    $this->updatedBy = $updatedBy;
    return $this;
}

public function getWeight(): ?float
{
    return $this->weight;
}

public function setWeight(?float $weight): self
{
    $this->weight = $weight;
}

```

```

        return $this;
    }
}
api/src/Ibs/Context/LabIoTGateway/Entity/PatientVitalsLatest.php
<?php

declare(strict_types=1);

namespace Ibs\Context\LabIoTGateway\Entity;

use ApiPlatform\Doctrine\Orm\Filter\SearchFilter;
use ApiPlatform\Metadata\ApiFilter;
use ApiPlatform\Metadata\ApiResource;
use ApiPlatform\Metadata\Get;
use ApiPlatform\Metadata\GetCollection;
use Ibs\Context\PatientManagement\Entity\Patient;
use Doctrine\ORM\Mapping as ORM;
use Ibs\Context\LabIoTGateway\State\PatientVitalsLatestBatchProvider;

#[ApiResource(
    operations: [
        new GetCollection(),
        new GetCollection(
            uriTemplate: '/patient_vitals_latests/batch',
            provider: PatientVitalsLatestBatchProvider::class
        ),
        new Get(),
    ]
)]
#[ApiFilter(SearchFilter::class, properties: ['patient' => 'exact'])]
#[ORM\Entity]
#[ORM\Table(
    name: 'patient_vitals_latest',
    uniqueConstraints: [
        new ORM\UniqueConstraint(name: 'uniq_patient_id', columns: ['patient_id'])
    ]
)]
class PatientVitalsLatest
{
    #[ORM\Id]
    #[ORM\GeneratedValue(strategy: 'IDENTITY')]
    #[ORM\Column(type: 'integer')]
    private ?int $id = null;

    #[ORM\OneToOne(targetEntity: Patient::class)]
    #[ORM\JoinColumn(name: 'patient_id', referencedColumnName: 'id', nullable: false, unique: true, onDelete: 'CASCADE')]

```

```

private Patient $patient;

#[ORM\Column(type: 'float', nullable: true)]
private ?float $hb = null;

#[ORM\Column(type: 'integer', nullable: true)]
private ?int $heartRate = null;

#[ORM\Column(type: 'integer', nullable: true)]
private ?int $systolicPressure = null;

#[ORM\Column(type: 'integer', nullable: true)]
private ?int $diastolicPressure = null;

#[ORM\Column(type: 'integer', nullable: true)]
private ?int $saturation = null;

#[ORM\Column(type: 'float', nullable: true)]
private ?float $weight = null;

#[ORM\Column(type: 'datetime')]
private \DateTimeInterface $lastUpdated;

public function __construct(Patient $patient)
{
    $this->patient = $patient;
    $this->lastUpdated = new \DateTime();
}

public function getId(): ?int
{
    return $this->id;
}

public function getPatient(): Patient
{
    return $this->patient;
}

public function getHb(): ?float
{
    return $this->hb;
}

public function setHb(?float $hb): self
{
    $this->hb = $hb;
    return $this;
}

```

```

public function getHeartRate(): ?int
{
    return $this->heartRate;
}

public function setHeartRate(?int $heartRate): self
{
    $this->heartRate = $heartRate;
    return $this;
}

public function getSystolicPressure(): ?int
{
    return $this->systolicPressure;
}

public function setSystolicPressure(?int $systolicPressure):
self
{
    $this->systolicPressure = $systolicPressure;
    return $this;
}

public function getDiastolicPressure(): ?int
{
    return $this->diastolicPressure;
}

public function setDiastolicPressure(?int $diastolicPressure
): self
{
    $this->diastolicPressure = $diastolicPressure;
    return $this;
}

public function getSaturation(): ?int
{
    return $this->saturation;
}

public function setSaturation(?int $saturation): self
{
    $this->saturation = $saturation;
    return $this;
}

public function getWeight(): ?float
{
    return $this->weight;
}

```

```

    public function setWeight(?float $weight): self
    {
        $this->weight = $weight;
        return $this;
    }

    public function getLastUpdated(): \DateTimeInterface
    {
        return $this->lastUpdated;
    }

    public function setLastUpdated(\DateTimeInterface $lastUpdated): self
    {
        $this->lastUpdated = $lastUpdated;
        return $this;
    }
}
frontend/src/modules/medicalHistory/components/MnoChart/MnoChart.script.js
import { Line } from 'vue-chartjs';
import { formatDate } from '@modules/shared/utils/formatters';
import {
    Chart as ChartJS,
    registerables
} from 'chart.js';

ChartJS.register(...registerables);

// Кастомный плагин для подписей вне диапазона
const customLabelsPlugin = {
    id: 'customLabels',
    afterDatasetsDraw(chart) {
        const { ctx } = chart;
        const meta = chart.getDatasetMeta(0);
        const mnoFrom = chart.options.mnoFrom;
        const mnoTo = chart.options.mnoTo;

        if (!meta || !meta.data) return;

        meta.data.forEach((point, index) => {
            const value = chart.data.datasets[0].data[index];
            if (value === null || value === undefined) return;

            const below = mnoFrom !== null && value < mnoFrom;
            const above = mnoTo !== null && value > mnoTo;
            if (!below && !above) return;

            const x = point.x;
            const y = below ? point.y + 15 : point.y - 10;

```

```

        ctx.font = 'bold 10px Arial';
        ctx.fillStyle = below ? '#2a5c98' : '#e74c3c';
        ctx.textAlign = 'center';
        ctx.fillText(value, x, y);
    });
}
};

export default {
  name: 'MnoChart',
  components: { Line },
  props: {
    data: { type: Array, required: true },
    mnoFrom: { type: Number, default: null },
    mnoTo: { type: Number, default: null },
  },
  data() {
    return {
      customLabelsPlugin,
      selectedRange: 'all',
      ranges: [
        { value: '1m', label: '1 мес.' },
        { value: '3m', label: '3 мес.' },
        { value: '6m', label: '6 мес.' },
        { value: '1y', label: '1 год' },
        { value: 'all', label: 'Всё' },
      ],
    };
  },
  computed: {
    chartData() {
      const d = this.prepareChartData();
      if (!d) return null;
      return { labels: d.labels, datasets: d.datasets };
    },
    chartOptions() {
      return {
        responsive: true,
        maintainAspectRatio: false,
        plugins: {
          legend: { position: 'bottom' },
        },
        scales: {
          y: {
            title: { display: true, text: 'МНО' },
            min: 0,
          },
          x: {
            title: { display: true, text: 'Дата' },
          },
        },
      };
    },
  },
};

```

```

        },
        mnoFrom: this.mnoFrom,
        mnoTo: this.mnoTo,
    };
},
},
methods: {
    changeRange(range) {
        this.selectedRange = range;
    },
    prepareChartData() {
        if (!this.data || this.data.length === 0) return null;

        let items = this.data
            .filter(item => item.inr !== '-' && !isNaN(parseFloat(item.inr)))
            .sort((a, b) => new Date(a.date) - new Date(b.date));

        if (items.length === 0) return null;

        if (this.selectedRange !== 'all') {
            const lastDate = items.reduce((max, item) => new Date(item.date) > max ? new Date(item.date) : max, new Date(0));
            if (isNaN(lastDate.getTime())) return null;

            const msMap = {
                '1m': 30 * 24 * 60 * 60 * 1000,
                '3m': 90 * 24 * 60 * 60 * 1000,
                '6m': 180 * 24 * 60 * 60 * 1000,
                '1y': 365 * 24 * 60 * 60 * 1000,
            };
            const limit = new Date(lastDate.getTime() - msMap[this.selectedRange]);
            items = items.filter(item => new Date(item.date) >= limit);
        }

        const labels = items.map(item => formatDate(item.date));
        const inrValues = items.map(item => parseFloat(item.inr));

        const datasets = [
            {
                label: 'MH0',
                data: inrValues,
                borderColor: '#3498db',
            }
        ]
    }
}

```

```

        backgroundColor: 'rgba(52, 152, 219, 0.1)',
        tension: 0.3,
        pointRadius: 3,
        pointHoverRadius: 5,
      },
    ];

    if (this.mnoFrom !== null && this.mnoFrom !== undefined) {
      datasets.push({
        label: `Нижняя граница (${this.mnoFrom})`,
        data: new Array(labels.length).fill(this.mnoFrom),
        borderColor: '#27ae60',
        borderDash: [5, 5],
        pointRadius: 0,
        fill: false,
      });
    }
    if (this.mnoTo !== null && this.mnoTo !== undefined) {
      datasets.push({
        label: `Верхняя граница (${this.mnoTo})`,
        data: new Array(labels.length).fill(this.mnoTo),
        borderColor: '#e74c3c',
        borderDash: [5, 5],
        pointRadius: 0,
        fill: false,
      });
    }

    return { labels, datasets };
  },
};

frontend/src/modules/medicalHistory/components/Pharmacogenetics/Pharmacogenetics
import { usePharmacogeneticsStore } from '@modules/medicalHistory/stores/pharmacogeneticsStore';

export default {
  name: 'Pharmacogenetics',
  props: {
    patientId: { type: [String, Number], required: true },
    drugIri: { type: String, default: null },
  },
  data() {
    return {
      store: usePharmacogeneticsStore(),
    };
  },
};

```

```

    },
    watch: {
      patientId: {
        immediate: true,
        handler(newId) {
          if (newId) {
            this.store.fetchPharmacogenetics(newId, this
            .drugIri);
          }
        },
      },
      drugIri(newIri) {
        if (this.patientId) {
          this.store.fetchPharmacogenetics(this.patientId,
          newIri);
        }
      },
    },
    methods: {
      getGenotypeLabel(marker) {
        return this.store.getGenotypeLabel(marker);
      },
    },
  },
};
frontend/src/modules/medicalHistory/components/VitalsCard/VitalsCard.scri
// frontend/src/modules/medicalHistory/components/VitalsCard/Vit
alsCard.script.js

import { usePatientVitalsLatestStore } from '@modules/medicalHi
story/stores/patientVitalsLatestStore';

export default {
  name: 'VitalsCard',
  props: {
    patientId: { type: [String, Number], required: true },
    treatmentId: { type: [String, Number], default: null },
  },
  data() {
    return {
      editing: false,
      saving: false,
      saveError: null,
      form: {
        hb: null,
        heartRate: null,
        systolicPressure: null,
        diastolicPressure: null,
        saturation: null,
        weight: null,
        recordDt: '',

```

```

        comment: '',
    },
    originalForm: null,
};
},
computed: {
    store() {
        return usePatientVitalsLatestStore();
    },
    latest() {
        return this.store.latest;
    },
    loading() {
        return this.store.loading;
    },
    error() {
        return this.store.error;
    },
    hasAnyData() {
        const l = this.latest;
        return l && (
            l.hb !== null || l.heartRate !== null || l.systolicPressure !== null ||
            l.diastolicPressure !== null || l.saturation !== null ||
            l.weight !== null
        );
    },
    lastUpdatedFormatted() {
        if (!this.latest?.lastUpdated) return null;
        return new Date(this.latest.lastUpdated).toLocaleString('ru-RU');
    },
},
watch: {
    patientId: {
        immediate: true,
        handler(newId) {
            if (newId) {
                this.store.fetchLatest(newId);
            }
        },
    },
},
},
methods: {
    startEditing() {
        this.form = {
            hb: this.latest?.hb ?? null,
            heartRate: this.latest?.heartRate ?? null,
            systolicPressure: this.latest?.systolicPressure ?? null,
            diastolicPressure: this.latest?.diastolicPressure ?? null
        };
    }
}

```

```

    },
    saturation: this.latest?.saturation ?? null,
    weight: this.latest?.weight ?? null,
    recordDt: this.formatDateTimeLocal(new Date()),
    comment: '',
  };
  this.originalForm = JSON.parse(JSON.stringify(this.form));
  this.editing = true;
  this.saveError = null;
},

cancelEditing() {
  if (this.originalForm) {
    this.form = JSON.parse(JSON.stringify(this.originalForm));
  }
  this.editing = false;
  this.saveError = null;
},

async save() {
  if (!this.validateForm()) return;

  this.saving = true;
  this.saveError = null;
  try {
    const payload = {
      patient: `/api/patients/${this.patientId}`,
      treatment: this.treatmentId ? `/api/treatments/${this.treatmentId}` : null,
      recordDt: this.form.recordDt,
      hb: this.form.hb,
      heartRate: this.form.heartRate,
      systolicPressure: this.form.systolicPressure,
      diastolicPressure: this.form.diastolicPressure,
      saturation: this.form.saturation,
      weight: this.form.weight,
      comment: this.form.comment || '',
    };
    await this.store.saveMeasurement(payload);
    await this.store.fetchLatest(this.patientId);
    this.editing = false;
  } catch (err) {
    console.error(err);
    this.saveError = err.response?.data?.detail || 'Ошибка с
    хранения.';
  } finally {
    this.saving = false;
  }
},

```

```

validateForm() {
  const { hb, heartRate, systolicPressure, diastolicPressure
, saturation, weight } = this.form;
  if (
    hb === null && heartRate === null && systolicPressure ==
= null &&
    diastolicPressure === null && saturation === null && wei
ght === null
  ) {
    this.saveError = 'Укажите хотя бы один показатель.';
    return false;
  }
  return true;
},

formatValue(value, unit) {
  return value !== null && value !== undefined ? `${value} $
{unit}` : '-';
},

formatDateTimeLocal(date) {
  const pad = (n) => String(n).padStart(2, '0');
  return `${date.getFullYear()}-${pad(date.getMonth() + 1)}-
${pad(date.getDate())}T${pad(date.getHours())}:${pad(date.ge
tMinutes())}`;
},
},
};
frontend/src/modules/shared/utils/vitalsHelpers.js
export function buildIndicators(testHistory, vitals) {
  const parts = [];

  // MHO
  if (testHistory?.mno !== null && testHistory?.mno !== undefi
ned) {
    parts.push(
      `

```

```

        `<span class="indicator-value">${vitals.hb}</span>` +
    n>` +
        `</span>`
    );
}
if (vitals.heartRate !== null && vitals.heartRate !== undefined) {
    parts.push(
        `<span class="indicator-item">` +
        `<span class="indicator-label">ЧСС</span>` +
        `<span class="indicator-value">${vitals.heartRate}</span>` +
        `</span>`
    );
}
if (vitals.systolicPressure !== null && vitals.diastolicPressure !== null) {
    parts.push(
        `<span class="indicator-item">` +
        `<span class="indicator-label">АД</span>` +
        `<span class="indicator-value">${vitals.systolicPressure}/${vitals.diastolicPressure}</span>` +
        `</span>`
    );
}
if (vitals.saturation !== null && vitals.saturation !== undefined) {
    parts.push(
        `<span class="indicator-item">` +
        `<span class="indicator-label">SpO2</span>` +
        `<span class="indicator-value">${vitals.saturation}%</span>` +
        `</span>`
    );
}
}

if (parts.length === 0) {
    return '-';
}

return `<span class="indicator-wrapper">${parts.join(' ')}</span>`;
}

/**
 * Обёртка для строк MedicalTable, где все поля (мно, hb, ...) лежат в одном объекте.
 * @param {object} row — объект с полями мно, hb, heartRate, systolicPressure, diastolicPressure, saturation, weight

```

```

    * @returns {string} HTML-строка с чипсами
    */
export function buildIndicatorsFromRow(row) {
    const testHistory = row.mno !== null ? { mno: row.mno } : null;
    const vitals = {
        hb: row.hb,
        heartRate: row.heartRate,
        systolicPressure: row.systolicPressure,
        diastolicPressure: row.diastolicPressure,
        saturation: row.saturation,
        weight: row.weight
    };
    return buildIndicators(testHistory, vitals);
}
frontend/src/modules/shared/api/pharmacogenetics.js
import apiClient from './client';

export const pharmacogeneticsApi = {
    getForPatient(patientId, config = {}) {
        return apiClient.get(`/patients/${patientId}/pharmacogenetics`, config);
    },
    createResult(data) {
        return apiClient.post('/patient_genetic_results', data);
    },
    updateResult(id, data) {
        return apiClient.put(`/patient_genetic_results/${id}`, data);
    },
    deleteResult(id) {
        return apiClient.delete(`/patient_genetic_results/${id}`);
    }
};
frontend/src/modules/shared/api/vitals.js
import apiClient from './client';

export const vitalsApi = {
    create(data) {
        return apiClient.post('/patient_vitals', data);
    },
    getLatest(patientId) {
        return apiClient.get('/patient_vitals_latests', {
            params: { patient: `/api/patients/${patientId}` }
        });
    },
    getBatch(patientIds) {
        return apiClient.get('/patient_vitals_latests/batch', {
            params: { 'patient_id[]': patientIds }
        });
    }
};

```

```

        });
    },
    getByTreatment(treatmentIri) {
        return apiClient.get('/patient_vitals', {
            params: {
                treatment: treatmentIri,
                order: { recordDt: 'asc' },
                itemsPerPage: 1000
            }
        });
    }
};
api/src/Ibs/Context/AICDSS/AICDSSBundle.php
<?php

namespace Ibs\Context\AICDSS;

use Symfony\Component\HttpKernel\Bundle\Bundle;

class AICDSSBundle extends Bundle
{
}
api/src/Ibs/Context/AICDSS/AiDosage/Dto/DosageRecommendation.php
<?php

declare(strict_types=1);

namespace Ibs\Context\AICDSS\AiDosage\Dto;

use ApiPlatform\Metadata\ApiResource;
use ApiPlatform\Metadata\Get;
use Ibs\Context\AICDSS\AiDosage\State\DosageRecommendationProvider;

#[ApiResource(
    shortName: 'Dosage',
    operations: [
        new Get(
            uriTemplate: '/dosage/recommendation',
            provider: DosageRecommendationProvider::class
        )
    ]
)]
class DosageRecommendation
{
    public array $variants = [];
    public string $explanation = '';
}
api/src/Ibs/Context/AICDSS/Command/SeedGeneticMarkersCommand.php

```

```

<?php

declare(strict_types=1);

namespace Ibs\Context\AICDSS\Command;

use Ibs\Context\TreatmentTherapy\Entity\Drug;
use Ibs\Context\AICDSS\Entity\GeneticMarker;
use Ibs\Context\AICDSS\Entity\GeneticMarkerValue;
use Ibs\Context\AICDSS\Entity\MarkerDrugRelation;
use Doctrine\ORM\EntityManagerInterface;
use Symfony\Component\Console\Attribute\AsCommand;
use Symfony\Component\Console\Command\Command;
use Symfony\Component\Console\Input\InputInterface;
use Symfony\Component\Console\Output\OutputInterface;

#[AsCommand(name: 'app:seed-genetic-markers')]
class SeedGeneticMarkersCommand extends Command
{
    private EntityManagerInterface $entityManager;

    public function __construct(EntityManagerInterface $entityManager)
    {
        parent::__construct();
        $this->entityManager = $entityManager;
    }

    protected function execute(InputInterface $input, OutputInterface $output): int
    {
        $output->writeln('Seeding genetic markers...');

        $markersData = [
            // CYP2C9*2
            [
                'geneSymbol' => 'CYP2C9_2',
                'fullName' => 'Цитохром P450 2C9, аллель *2',
                'rsId' => 'rs1799853',
                'description' => 'Генотип CYP2C9*2. Снижение активности фермента, повышение чувствительности к варфарину, риск кровотечений.',
                'values' => [
                    ['value' => 'CC', 'label' => 'CC (норма)', 'description' => 'Нормальная активность'],
                    ['value' => 'CT', 'label' => 'CT (гетерозигота)', 'description' => 'Снижение активности, повышенная чувствительность к варфарину'],
                    ['value' => 'TT', 'label' => 'TT (мутантная гомозигота)', 'description' => 'Значительное снижение активн

```

```

ости, высокая чувствительность, риск кровотечений'],
    ],
    // CYP2C9*3
    [
        'geneSymbol' => 'CYP2C9_3',
        'fullName' => 'Цитохром P450 2C9, аллель *3',
        'rsId' => 'rs1057910',
        'description' => 'Генотип CYP2C9*3. Снижение активности фермента, повышение чувствительности к варфарину, риск кровотечений.',
        'values' => [
            ['value' => 'AA', 'label' => 'AA (норма)', 'description' => 'Нормальная активность'],
            ['value' => 'AC', 'label' => 'AC (гетерозигота)', 'description' => 'Снижение активности, повышенная чувствительность к варфарину'],
            ['value' => 'CC', 'label' => 'CC (мутантная гомозигота)', 'description' => 'Значительное снижение активности, высокая чувствительность, риск кровотечений'],
        ],
    ],
    // VKORC1_3673
    [
        'geneSymbol' => 'VKORC1_3673',
        'fullName' => 'Витамин К эпоксидредуктаза, G3673A',
        'rsId' => 'rs9923231',
        'description' => 'Аллель А повышает чувствительность к варфарину (требуется меньшая доза)',
        'values' => [
            ['value' => 'GG', 'label' => 'GG (норма)', 'description' => 'Нормальная чувствительность к варфарину'],
            ['value' => 'GA', 'label' => 'GA (гетерозигота)', 'description' => 'Повышение чувствительности к варфарину, требуется меньшая доза'],
            ['value' => 'AA', 'label' => 'AA (мутантная гомозигота)', 'description' => 'Повышение чувствительности к варфарину, требуется меньшая доза'],
        ],
    ],
    // VKORC1_3730
    [
        'geneSymbol' => 'VKORC1_3730',
        'fullName' => 'Витамин К эпоксидредуктаза, G3730A',
        'rsId' => 'rs7294',
        'description' => 'Аллель А понижает чувствительность (резистентность), требуется более высокая доза варфарина',
    ],

```

```

        'values' => [
            ['value' => 'GG', 'label' => 'GG (норма)', '
description' => 'Нормальная чувствительность'],
            ['value' => 'GA', 'label' => 'GA (гетерозигота)', 'description' => 'Резистентность, требуется более высокая доза варфарина'],
            ['value' => 'AA', 'label' => 'AA (мутантная гомозигота)', 'description' => 'Резистентность, требуется более высокая доза варфарина'],
        ],
    ],
    // ABCB1
    [
        'geneSymbol' => 'ABCB1',
        'fullName' => 'ABC-транспортёр 1',
        'rsId' => null,
        'description' => 'Влияет на транспорт ПОАК (ривароксабан, апиксабан, дабигатран)',
        'values' => [
            ['value' => 'CC', 'label' => 'CC (норма)', 'description' => 'Нормальный транспорт ПОАК'],
            ['value' => 'CT', 'label' => 'CT (гетерозигота)', 'description' => 'Возможно изменение транспорта ПОАК'],
            ['value' => 'TT', 'label' => 'TT (мутантная гомозигота)', 'description' => 'Изменённый транспорт ПОАК'],
        ],
    ],
    // CYP3A5
    [
        'geneSymbol' => 'CYP3A5',
        'fullName' => 'Цитохром P450 3A5',
        'rsId' => null,
        'description' => 'Метаболизм апиксабана и ривароксабана',
        'values' => [
            ['value' => '*1/*1', 'label' => '*1/*1 (норма)', 'description' => 'Нормальный метаболизм'],
            ['value' => '*1/*3', 'label' => '*1/*3 (гетерозигота)', 'description' => 'Сниженный метаболизм апиксабана и ривароксабана'],
            ['value' => '*3/*3', 'label' => '*3/*3 (мутантная гомозигота)', 'description' => 'Значительно сниженный метаболизм апиксабана и ривароксабана'],
        ],
    ],
];

$markerRepo = $this->entityManager->getRepository(GeneticMarker::class);

```

```

        foreach ($markersData as $data) {
            $existing = $markerRepo->findOneBy(['geneSymbol' =>
$data['geneSymbol']]);
            if (!$existing) {
                $marker = new GeneticMarker();
                $marker->setGeneSymbol($data['geneSymbol']);
                $marker->setFullName($data['fullName']);
                $marker->setRsId($data['rsId']);
                $marker->setDescription($data['description']);
                $this->entityManager->persist($marker);

                foreach ($data['values'] as $val) {
                    $markerValue = new GeneticMarkerValue();
                    $markerValue->setMarker($marker);
                    $markerValue->setValue($val['value']);
                    $markerValue->setLabel($val['label']);
                    $markerValue->setDescription($val['descripti
on']);
                    $this->entityManager->persist($markerValue);
                }
                $output->writeln(sprintf(' Created marker %s wi
th %d values', $data['geneSymbol'], count($data['values'])))
;
            } else {
                $output->writeln(sprintf(' Marker %s already ex
ists', $data['geneSymbol']));
            }
        }
        $this->entityManager->flush();

        // Связи маркеров с препаратами
        $drugRepo = $this->entityManager->getRepository(Drug::cl
ass);
        $relationRepo = $this->entityManager->getRepository(Mark
erDrugRelation::class);

        $drugMappings = [
            'CYP2C9_2'      => ['варфарин', 'фенилин'],
            'CYP2C9_3'      => ['варфарин', 'фенилин'],
            'VKORC1_3673'   => ['варфарин', 'фенилин'],
            'VKORC1_3730'   => ['варфарин', 'фенилин'],
            'ABCB1'         => ['ривароксабан', 'апиксабан', 'даб
игатран'],
            'CYP3A5'        => ['апиксабан', 'ривароксабан'],
        ];

        foreach ($drugMappings as $geneSymbol => $drugNames) {
            $marker = $markerRepo->findOneBy(['geneSymbol' => $g
eneSymbol]);
            if (!$marker) {

```

```

        $output->writeln(sprintf(' ERROR: Marker %s not
found, skipping drug relations', $geneSymbol));
        continue;
    }

    foreach ($drugNames as $drugName) {
        $drug = $drugRepo->findOneBy(['nominative' => $d
rugName]);
        if (!$drug) {
            $output->writeln(sprintf(' WARNING: Drug "%
s" not found, skipping relation for %s', $drugName, $geneSym
bol));
            continue;
        }

        $existingRelation = $relationRepo->findOneBy([
            'marker' => $marker,
            'drug'    => $drug,
        ]);

        if (!$existingRelation) {
            $relation = new MarkerDrugRelation();
            $relation->setMarker($marker);
            $relation->setDrug($drug);
            $this->entityManager->persist($relation);
            $output->writeln(sprintf(' Created relation
: %s <-> %s', $geneSymbol, $drugName));
        } else {
            $output->writeln(sprintf(' Relation already
exists: %s <-> %s', $geneSymbol, $drugName));
        }
    }

    $this->entityManager->flush();

    $output->writeln('Seeding completed.');
```

return Command::SUCCESS;

```

    }
}

api/src/Ibs/Context/AICDSS/DependencyInjection/AICDSSExtension.php
<?php

namespace Ibs\Context\AICDSS\DependencyInjection;

use Symfony\Component\DependencyInjection\ContainerBuilder;
use Symfony\Component\DependencyInjection\Extension\Extension;

class AICDSSExtension extends Extension

```

```

{
    public function load(array $configs, ContainerBuilder $container): void
    {
        // Placeholder extension for AICDSS (AI Clinical Decision Support System) context.
        // Future context-specific service configuration can be added here.
    }
}
api/src/Ibs/Context/AICDSS/Pharmacogenetics/Dto/PharmacogeneticsResponse.php

declare(strict_types=1);

namespace Ibs\Context\AICDSS\Pharmacogenetics\Dto;

use ApiPlatform\Metadata\ApiResource;
use ApiPlatform\Metadata\GetCollection;
use Ibs\Context\AICDSS\Pharmacogenetics\State\PharmacogeneticsProvider;

#[ApiResource(
    operations: [
        new GetCollection(
            uriTemplate: '/patients/{patientId}/pharmacogenetics',
            provider: PharmacogeneticsProvider::class
        )
    ]
)]
class PharmacogeneticsResponse
{
    public array $markers = [];
}
api/src/Ibs/Context/LabIoTGateway/DependencyInjection/LabIoTGatewayExtension.php

namespace Ibs\Context\LabIoTGateway\DependencyInjection;

use Symfony\Component\DependencyInjection\ContainerBuilder;
use Symfony\Component\DependencyInjection\Extension\Extension;

class LabIoTGatewayExtension extends Extension
{
    public function load(array $configs, ContainerBuilder $container): void
    {
        // Placeholder extension for LabIoTGateway context.
        // Future context-specific service configuration can be

```

```

        added here.
    }
}
api/src/Ibs/Context/LabIoTGateway/EventListener/PatientVitalsAuditListene
<?php

declare(strict_types=1);

namespace Ibs\Context\LabIoTGateway\EventListener;

use Ibs\Context\LabIoTGateway\Entity\PatientVitals;
use Doctrine\Bundle\DoctrineBundle\Attribute\AsDoctrineListener;
use Doctrine\ORM\Event\PrePersistEventArgs;
use Doctrine\ORM\Event\PreUpdateEventArgs;
use Doctrine\ORM\Events;
use Symfony\Component\Security\Core\Authentication\Token\Storage
    \TokenStorageInterface;
use Symfony\Component\Security\Core\User\UserInterface;

#[AsDoctrineListener(event: Events::prePersist)]
#[AsDoctrineListener(event: Events::preUpdate)]
class PatientVitalsAuditListener
{
    public function __construct(
        private TokenStorageInterface $tokenStorage,
    ) {}

    public function prePersist(PrePersistEventArgs $args): void
    {
        $entity = $args->getObject();
        if (!$entity instanceof PatientVitals) {
            return;
        }
        $username = $this->getCurrentUsername();
        $entity->setCreatedBy($username);
        $entity->setUpdatedBy($username);
    }

    public function preUpdate(PreUpdateEventArgs $args): void
    {
        $entity = $args->getObject();
        if (!$entity instanceof PatientVitals) {
            return;
        }
        $entity->setUpdatedBy($this->getCurrentUsername());
    }

    private function getCurrentUsername(): ?string
    {
        $token = $this->tokenStorage->getToken();
    }
}

```

```

        if (null === $token) {
            return null;
        }
        $user = $token->getUser();
        return ($user instanceof UserInterface) ? $user->getUser
Identifier() : null;
    }
}
api/src/Ibs/Context/LabIoTGateway/EventListener/PatientVitalsListener.php
<?php

declare(strict_types=1);

namespace Ibs\Context\LabIoTGateway\EventListener;

use Ibs\Context\LabIoTGateway\Entity\PatientVitals;
use Ibs\Context\LabIoTGateway\Service\PatientVitalsSyncService;
use Doctrine\Bundle\DoctrineBundle\Attribute\AsDoctrineListener;
use Doctrine\ORM\Event\PostPersistEventArgs;
use Doctrine\ORM\Event\PostUpdateEventArgs;
use Doctrine\ORM\Events;

#[AsDoctrineListener(event: Events::postPersist)]
#[AsDoctrineListener(event: Events::postUpdate)]
class PatientVitalsListener
{
    public function __construct(
        private PatientVitalsSyncService $syncService,
    ) {}

    public function postPersist(PostPersistEventArgs $args): void
    {
        $this->syncIfVitals($args->getObject());
    }

    public function postUpdate(PostUpdateEventArgs $args): void
    {
        $this->syncIfVitals($args->getObject());
    }

    private function syncIfVitals(object $entity): void
    {
        if ($entity instanceof PatientVitals) {
            $this->syncService->syncFromVitals($entity);
        }
    }
}
api/src/Ibs/Context/LabIoTGateway/LabIoTGatewayBundle.php
<?php

```

```

namespace Ibs\Context\LabIoTGateway;

use Symfony\Component\HttpKernel\Bundle\Bundle;

class LabIoTGatewayBundle extends Bundle
{
}
api/src/Ibs/Context/LabIoTGateway/Validator/Constraints/AtLeastOneVitalSi
<?php

declare(strict_types=1);

namespace Ibs\Context\LabIoTGateway\Validator\Constraints;

use Symfony\Component\Validator\Constraint;
use Attribute;

#[Attribute]
class AtLeastOneVitalSign extends Constraint
{
    public string $message = 'Необходимо указать хотя бы один ви
тальный показатель.';

    public function getTargets(): string
    {
        return self::CLASS_CONSTRAINT;
    }
}
api/src/Ibs/Context/LabIoTGateway/Validator/Constraints/AtLeastOneVitalSi
<?php

declare(strict_types=1);

namespace Ibs\Context\LabIoTGateway\Validator\Constraints;

use Ibs\Context\LabIoTGateway\Entity\PatientVitals;
use Symfony\Component\Validator\Constraint;
use Symfony\Component\Validator\ConstraintValidator;
use Symfony\Component\Validator\Exception\UnexpectedTypeExceptio
n;

class AtLeastOneVitalSignValidator extends ConstraintValidator
{
    public function validate($value, Constraint $constraint): vo
id
    {
        if (!$constraint instanceof AtLeastOneVitalSign) {
            throw new UnexpectedTypeException($constraint, AtLea
stOneVitalSign::class);

```

```

    }

    if (!$value instanceof PatientVitals) {
        throw new UnexpectedTypeException($value, PatientVitals::class);
    }

    if ($value->getHb() === null &&
        $value->getHeartRate() === null &&
        $value->getSystolicPressure() === null &&
        $value->getDiastolicPressure() === null &&
        $value->getSaturation() === null &&
        $value->getWeight() === null
    ) {
        $this->context->buildViolation($constraint->message)
            ->addViolation();
    }
}
}

frontend/src/modules/medicalHistory/components/MnoChart/MnoChart.vue
<template src="./MnoChart.template.html"></template>
<script src="./MnoChart.script.js"></script>
<style scoped src="./MnoChart.style.css"></style>
frontend/src/modules/medicalHistory/components/Pharmacogenetics/Pharmacogenetics.vue
<template src="./Pharmacogenetics.template.html"></template>
<script src="./Pharmacogenetics.script.js"></script>
<style scoped src="./Pharmacogenetics.style.css"></style>
frontend/src/modules/medicalHistory/components/VitalsCard/VitalsCard.vue
<template src="./VitalsCard.template.html"></template>
<script src="./VitalsCard.script.js"></script>
<style scoped src="./VitalsCard.style.css"></style>

```